

LASSO Cellphone Addiction Analysis Proposal

Smartphone Addiction Project — Initial Results & Next Steps

Jonathan De La Torre

2026-06-10

Table of contents

1 Overview — What I’m Proposing	1
2 The Data	1
3 What We Predict (and Why)	2
4 How LASSO Works (Quick Version)	2
5 The Methods, Explained	3
5.1 Dial 1: What gets selected — plain vs group	3
5.2 Dial 2: What type of outcome — multinomial vs Gaussian	3
5.3 Two dials at a glance	3
5.4 Everything I ran so far	4
6 Results So Far	12
6.1 Master comparison — all 12 models	12
6.2 Gaussian regression — our main result	13
6.3 The plain read	13
7 Things to Keep in Mind	14
8 Where the Group Can Take It Next	14

i Reproducibility note

All tables below are produced by executing the embedded R code at render time. Results may vary slightly between renders because 10-fold cross-validation inside `glmnet` and `grpreg` is not fully deterministic, even with `set.seed(123)`.

1 Overview — What I’m Proposing

Hey team — here’s where I landed on our smartphone-addiction analysis, and what I think we can build on together toward the paper.

The goal: find out which survey scales best predict overall smartphone-addiction severity, using our dataset of **1,619** people and six validated instruments (NMP, SAPS, SPAI, MPAS, MPATS, SAS) plus demographics.

I’ve run an initial LASSO analysis (all the code and numbers below are live — they re-run when you render this doc). This write-up is my proposal for how we approach the project going forward: same data, same core question, room for all of us to extend it.

2 The Data

Each person answered items from six scale families plus demographics. Here’s a quick cheat sheet:

Scale	Abbreviation	What it measures
Nomophobia	NMP	Fear of being without one’s phone
Smartphone Addiction	SAPS	Tendency toward addictive phone use
Proneness Smartphone Addiction Inventory	SPAI	Symptom inventory for phone addiction
Mobile Phone Addiction Scale	MPAS	Addiction-related behaviors and feelings
Mobile Phone Addiction Tendency Scale	MPATS	Tendency toward problematic use
Smartphone Addiction Scale	SAS	Cognitive and behavioral addiction symptoms

We also have age, gender, and grade. After dropping rows with any missing answers, we’re left with **1,619** complete cases.

3 What We Predict (and Why)

Our main outcome is **SPAI total** — the sum of all 20 SPAI items. It’s a continuous severity score (roughly 20 to 80).

We **leave the SPAI items out of the predictors** because they *are* the score. Using them to predict the total would be circular — like predicting someone’s final exam score from the individual exam questions you already added up.

Instead, we ask: **do the other scales (nomophobia, proneness, MPAS, MPATS, SAS) and demographics track how severe someone’s SPAI total is?** That’s the question we can take into the paper.

4 How LASSO Works (Quick Version)

LASSO is a regression method that **automatically drops predictors that don’t help** and keeps the ones that do. It does this by shrinking coefficients toward zero; unhelpful ones get shrunk all the way to zero.

The penalty strength is controlled by **lambda**:

- **lambda.min** — best predictive fit on average (lowest cross-validation error).
- **lambda.1se** — the simplest model that’s nearly as good (within one standard error of the best). Think: “almost the same R^2 , but fewer variables.”

LASSO automatically picks which items matter — we don’t hand-select them beforehand.

5 The Methods, Explained

The naming gets confusing, so here’s the plain version. There are really **two independent choices**, not three competing “methods.”

5.1 Dial 1: What gets selected — plain vs group

Plain LASSO (`glmnet`) decides **item by item**. It might keep one SAPS question and drop another from the same scale.

Group LASSO (`grpreg`) is told that each scale is a **group**. It keeps or drops **whole scales at once** — either all of SAPS stays, or all of it goes.

- Plain answers: *which individual items matter?*
- Group answers: *which whole scales matter?*

In my results, group LASSO kept **many more** variables without much gain in R^2 . For the paper we lean toward **plain LASSO at lambda.1se** — a deliberate parsimony choice that keeps far fewer variables for nearly the same fit.

5.2 Dial 2: What type of outcome — multinomial vs Gaussian

This is set by `family =` in the model. It's a **separate dial** from plain vs group.

- `family = "multinomial"` — predict an **unordered category**. We used this when predicting a single SPAI item (SPAI_12) as one of four response options.
- `family = "gaussian"` — predict a **continuous number**. We used this for **SPAI total**.

Important: “**Gaussian LASSO**” is **not a third method**. It's just plain-or-group LASSO applied to a numeric outcome. You still choose plain *or* group; Gaussian only tells the model you're predicting a number instead of a category.

5.3 Two dials at a glance

	Predict a category (multinomial)	Predict a number (Gaussian)
Drop items individually (Plain LASSO)	Plain LASSO → SPAI_12 class	Plain LASSO → SPAI total
Drop whole scales (Group LASSO)	Group LASSO → SPAI_12 class	Group LASSO → SPAI total

Anchor line: plain vs group = drop items or drop whole scales. Multinomial vs Gaussian = predict a category or a number. These two choices are **independent** — you mix and match them.

5.4 Everything I ran so far

All models share **one 80/20 train/test split** (same people held out every time), so the numbers are comparable.

Classification side (multinomial family — predicting SPAI_12 as a 4-level category):

- Plain and Group LASSO, each in a few predictor-set variants (all predictors, SPAI items removed, SPAI + SAS removed).
- Eight models total. I report balanced accuracy and Kappa on the test set.

SPAI_12 is imbalanced — the rarest category is only about 6% of responses:

```
dat_raw <- read.csv(data_csv("DATA_Problematic Mobile Phone Use CAT.csv"))
dat_raw <- dat_raw[complete.cases(dat_raw), ]

class_tbl <- as.data.frame(table(SPAI_12 = dat_raw$SPAI_12))
class_tbl$Percent <- round(100 * class_tbl$Freq / sum(class_tbl$Freq), 1)
kable(class_tbl, align = c("l", "r", "r"))
```

Table 1: Distribution of SPAI_12 response categories (complete cases only).

SPAI_12	Freq	Percent
1	262	16.2
2	577	35.6
3	688	42.5
4	92	5.7

Regression side (Gaussian family — predicting **SPAI total**):

- Plain and Group LASSO, each at `lambda.min` and `lambda.1se`.
- Four models. This is our **main result** for the paper.

That's **12 models** in one comparison table. The code below loads the data, defines helper functions, fits everything, and builds the tables.

```
dat <- read.csv(data_csv("DATA_Problematic Mobile Phone Use CAT.csv"))
dat <- dat[complete.cases(dat), ]

dat$gender <- as.factor(dat$gender)
dat$grade <- as.factor(dat$grade)
dat$SPAI_total <- rowSums(dat[, paste0("SPAI_", 1:20)])
dat$SPAI_12 <- as.factor(dat$SPAI_12)
dat$SPAI_12_num <- as.numeric(dat$SPAI_12)

spai_drop <- setdiff(grep("^SPAI_", names(dat), value = TRUE), "SPAI_12")
sas_drop <- grep("^SAS_", names(dat), value = TRUE)

train_index <- createDataPartition(dat$SPAI_12, p = 0.80, list = FALSE)
y_class_test <- dat$SPAI_12[-train_index]

cat("Training n =", length(train_index), "| Test n =", length(y_class_test), "\n")
```

Training n = 1297 | Test n = 322

```
class_metrics <- function(pred, y) {
  pred <- factor(pred, levels = levels(y))
  cm <- confusionMatrix(pred, y)
  bc <- as.data.frame(cm$byClass)
  list(
    Accuracy = as.numeric(cm$overall["Accuracy"]),
    Balanced_Accuracy = mean(bc$Sensitivity, na.rm = TRUE),
    Kappa = as.numeric(cm$overall["Kappa"])
  )
}
```

```

reg_metrics <- function(y_true, y_pred) {
  ss_res <- sum((y_true - y_pred)^2)
  ss_tot <- sum((y_true - mean(y_true))^2)
  list(
    R2 = 1 - ss_res / ss_tot,
    RMSE = sqrt(mean((y_true - y_pred)^2))
  )
}

assign_group <- function(nm) {
  if (grepl("^gender", nm)) return("gender")
  if (grepl("^grade", nm)) return("grade")
  if (nm == "age") return("age")
  if (grepl("^NMP_", nm)) return("NMP")
  if (grepl("^SAPS_", nm)) return("SAPS")
  if (grepl("^SPAI_", nm)) return("SPAI")
  if (grepl("^MPAS_", nm)) return("MPAS")
  if (grepl("^MPATS", nm)) return("MPATS")
  if (grepl("^SAS_CA", nm)) return("SAS_CA")
  if (grepl("^SAS_C", nm)) return("SAS_C")
  "other"
}

make_class_mm <- function(dat, drop_cols = character(0)) {
  d <- dat[, !names(dat) %in% c("SPAI_12_num", "SPAI_total", drop_cols), drop = FALSE]
  x <- model.matrix(SPAI_12 ~ ., data = d)[, -1]
  list(
    x = x,
    y = d$SPAI_12,
    group = as.numeric(factor(sapply(colnames(x), assign_group)))
  )
}

make_reg_mm <- function(dat) {
  spai_items <- paste0("SPAI_", 1:20)
  d <- dat[, !names(dat) %in% c(spai_items, "SPAI_12_num", "SPAI_total"), drop = FALSE]
  d$SPAI_total <- dat$SPAI_total
  x <- model.matrix(SPAI_total ~ ., data = d)[, -1]
  list(
    x = x,
    y = d$SPAI_total,
    group = as.numeric(factor(sapply(colnames(x), assign_group)))
  )
}

plain_lasso_vars <- function(cv_fit, lambda_label, cols) {
  raw_coef <- coef(cv_fit, s = lambda_label)

```

```

if (is.list(raw_coef)) {
  vars <- unique(unlist(lapply(raw_coef, function(m) {
    mat <- as.matrix(m)
    rownames(mat)[mat[, 1] != 0]
  })))
} else {
  mat <- as.matrix(raw_coef)
  vars <- rownames(mat)[mat[, 1] != 0]
}
intersect(setdiff(vars, "(Intercept)"), cols)
}

group_lasso_vars <- function(cv_fit, lambda_val, cols) {
  coefs <- coef(cv_fit, lambda = lambda_val)
  intersect(names(coefs)[coefs != 0 & names(coefs) != "(Intercept)"], cols)
}

lambda_1se_grp <- function(cv_fit) {
  i <- which.min(cv_fit$cve)
  max(cv_fit$lambda[cv_fit$cve <= cv_fit$cve[i] + cv_fit$cvse[i]])
}

add_result <- function(results, row) {
  new_row <- data.frame(
    Model = row$Model,
    Part = row$Part,
    Variant = row$Variant,
    Outcome = row$Outcome,
    Lambda = row$Lambda,
    Balanced_Accuracy = row$Balanced_Accuracy,
    Kappa = row$Kappa,
    R2 = row$R2,
    RMSE = row$RMSE,
    N_selected = row$N_selected,
    stringsAsFactors = FALSE
  )
  rbind(results, new_row)
}

run_plain_class <- function(x, y, train_index, tag) {
  xtr <- x[train_index, , drop = FALSE]
  xte <- x[-train_index, , drop = FALSE]
  ytr <- y[train_index]
  yte <- y[-train_index]

  cv <- cv.glmnet(xtr, ytr, alpha = 1, family = "multinomial", nfolds = 10)
  sel <- plain_lasso_vars(cv, "lambda.min", colnames(x))

```

```

pred_a <- predict(cv, xte, s = "lambda.min", type = "class")

if (length(sel) > 0) {
  tr <- cbind(SPAI_12 = ytr, as.data.frame(xtr[, sel, drop = FALSE]))
  te <- cbind(SPAI_12 = yte, as.data.frame(xte[, sel, drop = FALSE]))
  pred_b <- predict(multinom(SPAI_12 ~ ., tr, trace = FALSE), te, type = "class")
} else {
  pred_b <- rep(levels(yte)[1], length(yte))
}

list(
  a = list(name = paste(tag, "A: LASSO"), pred = pred_a, n = length(sel), sel = sel),
  b = list(name = paste(tag, "B: Post-LASSO"), pred = pred_b, n = length(sel), sel = sel)
)
}

run_group_class <- function(x, y, group, train_index, tag) {
  xtr <- x[train_index, , drop = FALSE]
  xte <- x[-train_index, , drop = FALSE]
  ytr <- y[train_index]
  yte <- y[-train_index]
  ytr_num <- as.numeric(ytr)

  cv <- cv.gprreg(xtr, ytr_num, group, family = "gaussian", nfolds = 10)
  lam <- cv$lambda.min
  sel <- group_lasso_vars(cv, lam, colnames(x))
  pred_num <- as.numeric(predict(cv, xte, lambda = lam))
  pred_a <- factor(
    pmax(1, pmin(nlevels(yte), round(pred_num))),
    levels = levels(yte)
  )

  if (length(sel) > 0) {
    tr <- cbind(SPAI_12 = ytr, as.data.frame(xtr[, sel, drop = FALSE]))
    te <- cbind(SPAI_12 = yte, as.data.frame(xte[, sel, drop = FALSE]))
    pred_b <- predict(multinom(SPAI_12 ~ ., tr, trace = FALSE), te, type = "class")
  } else {
    pred_b <- rep(levels(yte)[1], length(yte))
  }

  list(
    a = list(name = paste(tag, "A: Group LASSO"), pred = pred_a, n = length(sel), sel = sel),
    b = list(name = paste(tag, "B: Post-Group-LASSO"), pred = pred_b, n = length(sel), sel = sel)
  )
}

run_gaussian <- function(x, y, train_index, tag, method = c("plain", "group"),

```



```

        group = NULL, lambda_label = "lambda.min") {
method <- match.arg(method)
xtr <- x[train_index, , drop = FALSE]
xte <- x[-train_index, , drop = FALSE]
ytr <- y[train_index]
yte <- y[-train_index]

if (method == "plain") {
  cv <- cv.glmnet(xtr, ytr, alpha = 1, family = "gaussian", nfolds = 10)
  if (lambda_label == "lambda.min") {
    pred <- as.numeric(predict(cv, xte, s = "lambda.min"))
    sel <- plain_lasso_vars(cv, "lambda.min", colnames(x))
    lam <- cv$lambda.min
  } else {
    pred <- as.numeric(predict(cv, xte, s = "lambda.1se"))
    sel <- plain_lasso_vars(cv, "lambda.1se", colnames(x))
    lam <- cv$lambda.1se
  }
} else {
  cv <- cv.gprpreg(xtr, ytr, group, family = "gaussian", nfolds = 10)
  if (lambda_label == "lambda.min") {
    lam <- cv$lambda.min
  } else {
    lam <- lambda_1se_grp(cv)
  }
  pred <- as.numeric(predict(cv, xte, lambda = lam))
  sel <- group_lasso_vars(cv, lam, colnames(x))
}

list(name = tag, pred = pred, yte = yte, n = length(sel), sel = sel, lambda = lam)
}

```

```

mm_full <- make_class_mm(dat)
mm_c1 <- make_class_mm(dat, spai_drop)
mm_c2 <- make_class_mm(dat, c(spai_drop, sas_drop))
mm_reg <- make_reg_mm(dat)

stopifnot(
  nrow(mm_full$x) == nrow(mm_c1$x),
  nrow(mm_full$x) == nrow(mm_c2$x),
  nrow(mm_full$x) == nrow(mm_reg$x)
)

```

```

results <- data.frame(
  Model = character(),
  Part = character(),
  Variant = character(),

```

```

Outcome = character(),
Lambda = character(),
Balanced_Accuracy = numeric(),
Kappa = numeric(),
R2 = numeric(),
RMSE = numeric(),
N_selected = integer(),
stringsAsFactors = FALSE
)

plain_full <- run_plain_class(mm_full$x, mm_full$y, train_index, "Plain")
plain_c1 <- run_plain_class(mm_c1$x, mm_c1$y, train_index, "Plain C1")
plain_c2 <- run_plain_class(mm_c2$x, mm_c2$y, train_index, "Plain C2")

plain_rows <- list(
  list(Model = "Plain A: LASSO (all predictors)", Variant = "A", m = plain_full$a),
  list(Model = "Plain B: Post-LASSO (all predictors)", Variant = "B", m = plain_full$b),
  list(Model = "Plain C1: LASSO (no SPAI predictors)", Variant = "C1", m = plain_c1$a),
  list(Model = "Plain C2: LASSO (no SPAI/SAS predictors)", Variant = "C2", m = plain_c2$a)
)

for (row in plain_rows) {
  met <- class_metrics(row$m$pred, y_class_test)
  results <- add_result(results, list(
    Model = row$Model,
    Part = "Plain LASSO",
    Variant = row$Variant,
    Outcome = "SPAI_12",
    Lambda = "lambda.min",
    Balanced_Accuracy = met$Balanced_Accuracy,
    Kappa = met$Kappa,
    R2 = NA,
    RMSE = NA,
    N_selected = row$m$n
  ))
}

grp_full <- run_group_class(mm_full$x, mm_full$y, mm_full$group, train_index, "Group")
grp_c1 <- run_group_class(mm_c1$x, mm_c1$y, mm_c1$group, train_index, "Group C1")
grp_c2 <- run_group_class(mm_c2$x, mm_c2$y, mm_c2$group, train_index, "Group C2")

group_rows <- list(
  list(Model = "Group A: Group LASSO (all predictors)", Variant = "A", m = grp_full$a),
  list(Model = "Group B: Post-Group-LASSO (all predictors)", Variant = "B", m = grp_full$b),
  list(Model = "Group C1: Group LASSO (no SPAI predictors)", Variant = "C1", m = grp_c1$a),
  list(Model = "Group C2: Group LASSO (no SPAI/SAS predictors)", Variant = "C2", m = grp_c2$a)
)

```

```

for (row in group_rows) {
  met <- class_metrics(row$m$pred, y_class_test)
  results <- add_result(results, list(
    Model = row$Model,
    Part = "Group LASSO",
    Variant = row$Variant,
    Outcome = "SPAI_12",
    Lambda = "lambda.min",
    Balanced_Accuracy = met$Balanced_Accuracy,
    Kappa = met$Kappa,
    R2 = NA,
    RMSE = NA,
    N_selected = row$m$n
  ))
}

```

Plain LASSO classification on SPAI_12 under three predictor sets (variant A: all predictors; C1: SPAI items excluded; C2: SPAI and SAS items excluded):

```

class_pred_sets <- results %>%
  filter(Part == "Plain LASSO", Variant %in% c("A", "C1", "C2")) %>%
  select(Model, Balanced_Accuracy, Kappa, N_selected) %>%
  mutate(
    Balanced_Accuracy = round(Balanced_Accuracy, 3),
    Kappa = round(Kappa, 3)
  )

kable(class_pred_sets, align = c("l", "r", "r", "r"))

```

Table 2: Plain LASSO classification: test-set performance under different predictor sets.

Model	Balanced_Accuracy	Kappa	N_selected
Plain A: LASSO (all predictors)	0.531	0.441	58
Plain C1: LASSO (no SPAI predictors)	0.484	0.396	41
Plain C2: LASSO (no SPAI/SAS predictors)	0.423	0.265	58

```

gauss_models <- list(
  run_gaussian(mm_reg$x, mm_reg$y, train_index,
    "Gaussian Plain | lambda.min", "plain", lambda_label = "lambda.min"),
  run_gaussian(mm_reg$x, mm_reg$y, train_index,
    "Gaussian Plain | lambda.1se", "plain", lambda_label = "lambda.1se"),
  run_gaussian(mm_reg$x, mm_reg$y, train_index,
    "Gaussian Group | lambda.min", "group",
    group = mm_reg$group, lambda_label = "lambda.min"),
  run_gaussian(mm_reg$x, mm_reg$y, train_index,

```

```

      "Gaussian Group | lambda.1se", "group",
      group = mm_reg$group, lambda_label = "lambda.1se")
)

gauss_rows <- list(
  list(Model = "Gaussian Plain | lambda.min", Variant = "Plain", Lambda = "lambda.min"),
  list(Model = "Gaussian Plain | lambda.1se", Variant = "Plain", Lambda = "lambda.1se"),
  list(Model = "Gaussian Group | lambda.min", Variant = "Group", Lambda = "lambda.min"),
  list(Model = "Gaussian Group | lambda.1se", Variant = "Group", Lambda = "lambda.1se")
)

for (i in seq_along(gauss_models)) {
  m <- gauss_models[[i]]
  met <- reg_metrics(m$ytest, m$pred)
  results <- add_result(results, list(
    Model = gauss_rows[[i]]$Model,
    Part = "Gaussian LASSO",
    Variant = gauss_rows[[i]]$Variant,
    Outcome = "SPAI_total",
    Lambda = gauss_rows[[i]]$Lambda,
    Balanced_Accuracy = NA,
    Kappa = NA,
    R2 = met$R2,
    RMSE = met$RMSE,
    N_selected = m$n
  ))
}

stopifnot(nrow(results) == 12)

```

6 Results So Far

6.1 Master comparison — all 12 models

```

results_display <- results %>%
  mutate(
    Balanced_Accuracy = ifelse(is.na(Balanced_Accuracy), NA, round(Balanced_Accuracy, 3)),
    Kappa = ifelse(is.na(Kappa), NA, round(Kappa, 3)),
    R2 = ifelse(is.na(R2), NA, round(R2, 3)),
    RMSE = ifelse(is.na(RMSE), NA, round(RMSE, 2))
  )

kable(
  results_display,

```

```

booktabs = TRUE,
longtable = TRUE,
caption = paste0(
  "All models on the shared test set (n = ", length(y_class_test),
  "). Classification rows use balanced accuracy and Kappa; Gaussian rows use R2 and RMSE."
)
)

```

Table 3: All models on the shared test set (n = 322). Classification rows use balanced accuracy and Kappa; Gaussian rows use R² and RMSE.

Model	Part	Vari- ant	Out- come	Lambda	Bal- anced_Ac- curacy	Kappa	R ²	RMSE	N_se- lected
Plain A: LASSO (all predictors)	Plain LASSO	A	SPAI_12	lambda.min	0.531	0.441	NA	NA	58
Plain B: Post-LASSO (all predictors)	Plain LASSO	B	SPAI_12	lambda.min	0.545	0.426	NA	NA	58
Plain C1: LASSO (no SPAI predictors)	Plain LASSO	C1	SPAI_12	lambda.min	0.484	0.396	NA	NA	41
Plain C2: LASSO (no SPAI/SAS predictors)	Plain LASSO	C2	SPAI_12	lambda.min	0.423	0.265	NA	NA	58
Group A: Group LASSO (all predictors)	Group LASSO	A	SPAI_12	lambda.min	0.551	0.463	NA	NA	111
Group B: Post-Group-LASSO (all predictors)	Group LASSO	B	SPAI_12	lambda.min	0.490	0.344	NA	NA	111
Group C1: Group LASSO (no SPAI predictors)	Group LASSO	C1	SPAI_12	lambda.min	0.431	0.311	NA	NA	91
Group C2: Group LASSO (no SPAI/SAS predictors)	Group LASSO	C2	SPAI_12	lambda.min	0.355	0.233	NA	NA	67
Gaussian Plain lambda.min	Gaussian LASSO	Plain	SPAI_tol	lambda.min	NA	NA	0.766	5.08	64
Gaussian Plain lambda.1se	Gaussian LASSO	Plain	SPAI_tol	lambda.1se	NA	NA	0.752	5.23	29
Gaussian Group lambda.min	Gaussian LASSO	Group	SPAI_tol	lambda.min	NA	NA	0.764	5.10	92
Gaussian Group lambda.1se	Gaussian LASSO	Group	SPAI_tol	lambda.1se	NA	NA	0.751	5.25	63

6.2 Gaussian regression — our main result

```
gauss_tbl <- results %>%  
  filter(Outcome == "SPAI_total") %>%  
  select(Model, Lambda, R2, RMSE, N_selected) %>%  
  mutate(R2 = round(R2, 3), RMSE = round(RMSE, 2))  
  
kable(gauss_tbl, align = c("l", "l", "r", "r", "r"))
```

Table 4: Gaussian LASSO models predicting SPAI total from non-SPAI scales (test set).

Model		Lambda	R2	RMSE	N_selected
Gaussian Plain	lambda.min	lambda.min	0.766	5.08	64
Gaussian Plain	lambda.1se	lambda.1se	0.752	5.23	29
Gaussian Group	lambda.min	lambda.min	0.764	5.10	92
Gaussian Group	lambda.1se	lambda.1se	0.751	5.25	63

6.3 The plain read

Test R^2 is around **0.77** — so the other scales explain about **three-quarters** of the variation in SPAI severity on people the model hasn't seen.

SAPS (addiction proneness) and **SAS** do most of the work. **Nomophobia (NMP)** barely shows up — once proneness and SAS are in the model, fear-of-disconnection items don't add much.

For the paper we recommend **plain LASSO at lambda.1se** (R^2 **0.75** in this render) because it keeps far fewer variables for nearly the same R^2 — a parsimony choice, not a hunt for the single highest decimal. Group LASSO at `lambda.min` lands in a similar R^2 range but with many more predictors in the mix.

7 Things to Keep in Mind

- **The scales overlap.** They all measure facets of problematic phone use, so a high R^2 partly means “they move together” — convergent validity, not proof that one scale *causes* another.
- **Numbers wobble a bit.** Cross-validation picks the penalty, and fold assignment isn't perfectly repeatable. Treat the results as solid but not exact to the third decimal.
- **This is our sample.** These findings describe the 1,619 people in this dataset. We'd need a new sample to know how well they generalize.

8 Where the Group Can Take It Next

Here are directions we could split up and develop for the paper — totally open, not assignments:

- **External validation** — test the model on a fresh sample and see if SAPS/SAS still dominate.
- **Ordinal models** — try an ordered-outcome version for the categorical SPAI_12 analysis instead of plain multinomial.
- **Write up the SAPS/SAS story** — why proneness and the SAS scale track SPAI total so closely, and what that means substantively.
- **Robustness checks** — different random seeds or train/test splits to confirm the R^2 and variable-selection pattern holds up.
- **Why nomophobia drops out** — dig into whether NMP is redundant with SAPS/SAS or just weaker on its own.

That's the foundation. Happy to walk through any of the code live — just render this doc and everything re-runs.